Table of Contents

End-to-End AI Application for 3D Scene Reconstruction

MLOps-Compliant Architecture & Implementation Guide

Table of Contents

1. Problem Definition & Success Metrics
2. System Architecture Overview
3. Stage 1 — Data Collection & Validation
4. Stage 2 — Data Preprocessing & Feature Engineering
5. Stage 3 — Model Development & Training
6. Stage 4 — Inference Pipeline (Production)
7. Stage 5 — Model Deployment
8. Stage 6 — Monitoring & Maintenance
9. Technology Stack & Tooling
10. Repository & Project Structure
11. CI/CD Pipeline
12. Security & Compliance

---

1. Problem Definition & Success Metrics

Business Problem

Given a set of multi-view images of a scene, the application must recover the 3D structure of that scene by estimating each camera's rotation matrix (R) and translation vector (t) with high accuracy. This enables applications in AR/VR, robotics, autonomous driving, cultural heritage digitization, and surveying.

ML Metrics

Metric	Definition	Target
mAA (mean Average Accuracy)	% cameras registered within a set of translation thresholds	≥ 50% on public benchmark
Reprojection Error	Mean pixel error of	≤ 1.0 px

Metric	Definition	Target
	triangulated 3D points	
Registration Rate	Fraction of images successfully registered in reconstruction	≥ 90%
Inlier Ratio	RANSAC inliers / total matches	≥ 25%

Business / Operational Metrics

Metric	Target
End-to-end inference latency per scene (5 images)	≤ 5 minutes on dual-GPU (T4 × 2)
API response time (health check)	≤ 200 ms
Model artifact size	≤ 10 GB (all weights combined)
Throughput	≥ 1 scene / minute at peak

2. System Architecture Overview

The application follows a modular, pipeline-based design where each stage is independently versioned, containerised, and monitored. The six major runtime components are:

```
[Image Store] → [Shortlist Generator] → [Pairwise Matcher]
     → [Geometric Verifier] → [SfM Reconstructor] → [Output Store]
```

Each component maps to a clearly defined Python class in the codebase (e.g., `ShortlistGenerator`, `MASt3RHybridMatcher`, `IMC2025Pipeline`). The entire flow is orchestrated by `IMC2025Pipeline.run_scene()` and distributed across GPUs via `torchrun` with `DistConfig`.

Component Responsibility Map

Component	Source Class	Responsibility
Data Schema	`DataSchema` / `IMC2025TestData`	Load CSVs, resolve image paths, format output keys
Shortlist Generator	`ShortlistGeneratorConfig` / `EnsembleShortlistGenerator`	Produce candidate image pairs using global descriptors
Global Descriptors	`MASt3RRetrievalASMKShortlistGenerator`, `DINOv2`, `ISC`, `SPoC`	Encode images into compact vectors for retrieval

Component	Source Class	Responsibility
Pairwise Matcher	`MASt3RHybridMatcher`	Produce semi-dense matches + keypoint-level matches
Geometric Verifier	`verify_matches` / `GeometricVerificationConfig`	RANSAC-based fundamental matrix estimation
Reconstructor	`pycolmap.incremental_mapping`	Incremental SfM, outputs R and t per camera
Storage Abstractions	`InMemoryKeypointStorage`, `InMemoryMatchingStorage`	Thread-safe in-memory structures for matches and keypoints
Distributed Coordinator	`DistConfig`, `init_dist`, `split_df_by_dataset`	Multi-GPU data splitting and result merging

3. Stage 1 — Data Collection & Validation

3.1 Data Sources

The primary dataset is the **Image Matching Challenge 2025 (IMC25)** dataset, structured as:

```
data/
├── sample_submission.csv
├── train_labels.csv
├── train_thresholds.csv
├── test/
│   ├── ETs/
│   └── stairs/
└── train/
    ├── ETs/
    ├── amy_gardens/
    └── ...
```

Each row in `train_labels.csv` carries: `dataset`, `scene`, `image`, `rotation_matrix` (9 values, semicolon-separated), and `translation_vector` (3 values).

3.2 Data Ingestion Pipeline

The `DataSchema` base class (`data_schema.py`) provides a uniform interface:

- `resolve_image_path(row)` — converts a CSV row to an absolute file path.

- `check_files_exist(drop=True)` — drops rows for missing files before training begins.
- `preprocess()` — called once at startup to validate integrity.
- `head(n)` — fast-commit mode that limits dataset to N rows.

Concrete implementations `IMC2025TestData` and `IMC2025TrainData` each override `build_image_relative_path()` to handle different directory layouts without changing downstream code.

3.3 Automated Validation Checks

Check	Implementation
File existence	`DataSchema.check_files_exist()`
Schema consistency	Pydantic models on every config (e.g., `SubmissionConfig`, `IMC2025PipelineConfig`)
Image readability	`cv2.imread` during `Scene.cache_all_images()` — `None` returns are logged
Rotation matrix validity	Post-reconstruction assertion in `get_best_reconstruction()`
NaN / Inf in outputs	`np.isnan` / `np.isinf` checks before writing `submission.csv`

3.4 DVC Data Versioning

All raw data directories and CSV manifests are tracked with **DVC** backed by local NAS or shared filesystem (no cloud). Commands:

```
dvc add data/train_labels.csv data/train/ data/test/
dvc push
git add data/.gitignore data/train_labels.csv.dvc
git commit -m "data: version IMC25 dataset v1.0"
```

---

4. Stage 2 — Data Preprocessing & Feature Engineering

4.1 Image-Level Preprocessing

Each image passes through one or more optional preprocessing steps before matching, all implemented as composable pipeline stages in `IMC2025Pipeline.run_scene()`:

Step	Config Class	Purpose
Orientation Normalization	`OrientationNormalizationConfig`	Correct rotation using `CheckOrientationHandler` (ResNeXt-50 classifier)
Deblurring	`DeblurringConfig`	FFTformer-based deblur for blurry frames (Laplacian variance threshold)
Segmentation	`SegmentationConfig`	Grounded-SAM mask generation to isolate foreground objects
Depth Estimation	`DepthEstimationConfig`	DepthAnything v2 for metric depth (used by downstream matching)

All preprocessed images are cached in `Scene.deblurred_images` and `Scene.depth_images` dictionaries, keeping original images untouched (`use_original=True` flag).

4.2 Feature Engineering

Global Descriptors (for image retrieval / shortlist generation):

Descriptor	Class	Dimension	Purpose
MASt3R-ASMK	`MASt3RRetrievalASMKRetriever`	variable	Vocabulary-tree retrieval with ASMK aggregation
MASt3R-SPoC	`MASt3RRetrievalSPoCGlobalDescriptorExtractor`	1024	Sum-pooled dense features
DINOv2	`DINOv2GlobalDescriptorExtractor`	768	Max-pooled ViT features
ISC	`ISCGlobalDescriptorExtractor`	256	Copy-detection descriptor

Descriptor	Class	Dimension	Purpose
SigLIP2	`SigLIP2GlobalDescriptorExtractor`	1152	Vision-language embedding

Local Features (for pairwise matching):

Feature	Class	Max Keypoints	Resize
ALIKED+LightGlue	`LightGlueALIKEDHandler`	4096	1280 px
SuperPoint (MagicLeap)	`MagicLeapSuperPointHandler`	4096	1600 px
MASt3R Dense	`MASt3RHybridMatcher`	all pixels / 8	512 px

4.3 Drift Baselines

During EDA, the following statistical baselines are recorded for later drift detection:

- Mean / std of descriptor L2 norms per model
- Distribution of match count per image pair
- Distribution of inlier ratios per scene type
- Image resolution distribution per dataset

These are logged to **MLflow** as run artifacts at the end of every training evaluation cycle.

---

5. Stage 3 — Model Development & Training

5.1 Pretrained Models Used

The application is primarily **inference-only** — all backbone models are pretrained and loaded from checkpoints. The configuration-driven design (`conf/models.yaml`) maps symbolic names to file paths, making model swapping trivial.

Model Key	Architecture	Source
`MAST3R`	AsymmetricMASt3R ViT-Large + DPT head	naver/mast3r

Model Key	Architecture	Source
`MAST3R_RETRIEVAL`	MASt3R + retrieval projection head	naver/mast3r
`ALIKED_LIGHTGLUE_N16`	ALIKED-N16 + LightGlue transformer	cvg/LightGlue
`MAGICLEAP_SUPERPOINT`	SuperPoint MLP	magicleap (research only)
`DINOV2_BASE`	DINOv2 ViT-Base	facebookresearch/dinov2
`ISC`	ISC21 ResNet	lyakaap/ISC21

5.2 Fine-tuning / Adaptation Strategy

While backbone weights are frozen, the following are configurable per experiment:

- Matching thresholds (`dense_match_threshold`, `sparse_match_threshold`)
- Subsampling factors (`dense_subsample`, `dense_pixel_tol`)
- Shortlist sizes (top-k per retrieval model)
- RANSAC parameters (`threshold`, `confidence`, `max_iters`)
- COLMAP mapper settings (`min_model_size`, `max_num_models`)

Each configuration is serialized as a YAML file (e.g., `conf/pipeline/imc2025/mast3rhybrid/mast3rhybrid-022-b.yaml`) and loaded via `SubmissionConfig.load_config_from_pipeline_config_string()`.

5.3 Experiment Tracking with MLflow

Every experiment run logs the following to MLflow:

```
# Pseudocode — runs inside evaluate_imc2025.py
with mlflow.start_run(run_name=conf_path.stem):
    mlflow.log_param("pipeline_type", conf.pipeline.type)
    mlflow.log_params(flatten_config(conf))        # all YAML params
    mlflow.log_metric("mAA", final_score)
    mlflow.log_metric("elapsed_sec", elapsed_time)
    for dataset, score in dataset_scores.items():
        mlflow.log_metric(f"mAA_{dataset}", score)
    mlflow.log_artifact(str(conf_path))            # config YAML
    mlflow.log_artifact("submission.csv")          # full predictions
```

Git commit hash is captured automatically via `mlflow.set_tag("git_commit", subprocess.check_output(["git", "rev-parse", "HEAD"]))`.

5.4 Hyperparameter Sensitivity Analysis

Key findings from the REPORT.md ablation studies:

Sparse Detectors	Shortlist Strategy	Public mAA
ALIKED(4k) + SP(4k)	ASMK + SPoC + DINOv2 + ISC	**52.64%**
ALIKED(4k) + DaD(4k)	ASMK + SPoC + DINOv2 + ISC	52.41%
GIMSP(4k) + ALIKED(4k)	ASMK(k=60) only	49.25%
SIFT(4k) only	ASMK + SPoC + DINOv2 + ISC	50.50%

6. Stage 4 — Inference Pipeline (Production)

6.1 Pipeline Execution Flow

The inference pipeline is invoked by `kernel.py::run()` and orchestrated by `IMC2025Pipeline`. The following sequence is executed per scene:

Step 1 — Scene Setup Load all images into memory via `Scene.cache_all_images()` (parallel `ProcessPoolExecutor`, max 4 GB cache).

Step 2 — Shortlist Generation `EnsembleShortlistGenerator` runs four retrievers in sequence: - MASt3R-ASMK: FPS-based pair selection (n=10, k=25) - MASt3R-SPoC: top-10 nearest neighbors - DINOv2: top-10 nearest neighbors - ISC: top-10 nearest neighbors

The union of all pairs is the final shortlist. `all_pairs_fallback_threshold=0` ensures all-pairs matching when the scene is tiny (≤ threshold images).

Step 3 — Point Tracking Matching `MASt3RHybridMatcher` is applied to each image pair in the shortlist: 1. MASt3R forward pass at size 512 → semi-dense matches via `fast_reciprocal_NNs`. 2. ALIKED keypoints (4096, resize 1280) are fed to MASt3R's local descriptor head. 3. SuperPoint keypoints (4096, resize 1600) are fed similarly. 4. All three match sets are merged via `fuse_matching_sets_late()`.

Step 4 — Geometric Verification `verify_matches()` runs COLMAP's exhaustive matcher (`pycolmap.match_exhaustive`) which applies RANSAC internally to estimate fundamental matrices and retain geometric inliers.

Step 5 — Incremental SfM Reconstruction `pycolmap.incremental_mapping()` builds a 3D model. Key options: `min_model_size=3`, `max_num_models=25`. Results from multiple sub-models are merged using Horn registration.

Step 6 — Output Serialization `results_to_submission_df()` formats R and t as semicolon-delimited strings and writes `submission.csv`.

6.2 Distributed Execution

For multi-GPU inference, `torchrun` with `--nproc_per_node 2` is used:

```
torchrun --nnodes 1 --nproc_per_node 2 --standalone \
  -m ns64_imc2025lib.kernel \
  -c config.yaml --env-name kernel --dist --kaggle-submit
```

`split_df_by_dataset()` partitions datasets across ranks. Each rank writes `submission_{rank}.csv`; rank 0 waits for all parts and merges them via `wait_all_submissions()`.

6.3 Determinism

`torch.use_deterministic_algorithms(True)` and `CUBLAS_WORKSPACE_CONFIG=':4096:8'` are set globally. `set_random_seed(seed=1234)` seeds NumPy, Python random, and PyTorch. This ensures every run with the same config and data produces bit-identical outputs.

---

7. Stage 5 — Model Deployment

7.1 Container Strategy

A three-container `docker-compose` setup is used:

```
# docker-compose.yml (summary)
services:
  api:
    build: ./api          # FastAPI wrapper around inference pipeline
    ports: ["8000:8000"]
    depends_on: [model_server]
    environment:
      - MODEL_SERVER_URL=http://model_server:8001
    healthcheck:
```

```yaml
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s

  model_server:
    build: ./model_server  # Loads MASt3R, ALIKED, DINOv2, ISC weights
    runtime: nvidia
    environment:
      - DEFAULT_MODEL_LIST_PATH=/app/conf/models.yaml
    volumes:
      - ./weights:/app/weights:ro
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8001/ready"]

  monitoring:
    image: prom/prometheus
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports: ["9090:9090"]
```

7.2 API Endpoints

The FastAPI service exposes:

Endpoint	Method	Description
`/health`	GET	Liveness check — returns 200 if process is alive
`/ready`	GET	Readiness check — returns 200 only when all model weights are loaded
`/reconstruct`	POST	Accept ZIP of images; run full pipeline; return `submission.csv`
`/reconstruct/async`	POST	Submit job; returns `job_id` for polling
`/jobs/{job_id}`	GET	Poll job status and retrieve result
`/metrics`	GET	Prometheus-compatible metrics endpoint

7.3 Model Versioning & Registry

MLflow Model Registry is used with three stages: **Staging**, **Production**, **Archived**.

```bash
# Register best model config after evaluation
mlflow models register \
  --run-id <RUN_ID> \
```

```
  --model-uri "runs:/<RUN_ID>/config" \
  --name "scene_reconstruction_pipeline"

# Promote to production after validation
mlflow models transition-stage \
  --name "scene_reconstruction_pipeline" \
  --version 3 \
  --stage Production
```

7.4 Rollback Strategy

A blue-green deployment pattern is used via Docker Swarm:

1. New model config is validated on Staging with 10% traffic.
2. If mAA on validation set drops > 5% from Production baseline, the release is blocked.
3. If a production failure is detected, `docker service update --rollback scene_api` reverts to the previous image within seconds.

---

8. Stage 6 — Monitoring & Maintenance

8.1 Performance Monitoring

The following metrics are exported to Prometheus from the `/metrics` endpoint:

Metric Name	Type	Description
`reconstruction_maa`	Gauge	mAA score on validation split (updated nightly)
`inference_latency_seconds`	Histogram	Per-scene wall-clock time
`match_count_per_pair`	Histogram	Number of matched keypoints
`inlier_ratio`	Gauge	RANSAC inlier fraction
`registered_images_ratio`	Gauge	Fraction of images successfully placed
`api_requests_total`	Counter	Total HTTP requests by endpoint
`api_errors_total`	Counter	4xx / 5xx responses

Grafana dashboards are pre-configured with alert rules:

- **Alert 1**: `api_errors_total rate > 5%` over 5-minute window → PagerDuty notification.

- **Alert 2**: `reconstruction_maa < 0.45` → Slack notification to ML team.
- **Alert 3**: `inference_latency_seconds p99 > 600` → auto-scale or throttle request queue.

8.2 Data Drift Detection

Statistical baselines computed during EDA are stored as MLflow artifacts. At every inference batch, a lightweight drift checker compares:

- **Descriptor norm distribution** (KS-test, threshold $p < 0.01$)
- **Match count distribution** (KS-test)
- **Image resolution distribution** (chi-squared test)

If drift is detected, an alert is raised and the input batch is flagged for human review before results are returned.

8.3 The Feedback Loop

When ground-truth camera poses become available (e.g., from survey instruments or human annotation), the following loop runs:

1. Ground-truth CSV is ingested and matched to corresponding `image_id` values.
2. `evaluate_imc2025.py` computes mAA against the new ground-truth.
3. MLflow logs the result as a new metric under the production run's parent experiment.
4. If mAA drops below the threshold, a retraining/reconfiguration ticket is automatically opened in the issue tracker.

8.4 Model Retraining Triggers

Trigger	Condition	Action
Scheduled	Weekly cron (Sunday 02:00 UTC)	Re-evaluate all registered configs on held-out validation split
Drift detected	KS-test $p < 0.01$ on descriptors	Alert + flag for manual config review
Performance decay	Production mAA < 0.45	Promote the next-best Staging model to Production

Trigger	Condition	Action
New data	New scenes added to training set	DVC pull + full re-evaluation cycle

9. Technology Stack & Tooling

Layer	Tool	Version / Notes
Language	Python	3.11
Deep Learning	PyTorch	2.5.1
Computer Vision	pycolmap, kornia, OpenCV	3.11.1, 0.8.x, 4.11.x
3D Reconstruction	MASt3R, DUSt3R, GLOMAP	naver/mast3r
Feature Matching	LightGlue, ALIKED, SuperPoint	cvg/LightGlue
Config Management	Pydantic v2, YAML	all configs validated at load time
Data Versioning	DVC	Git LFS for large model checkpoints
Experiment Tracking	MLflow	local tracking server on port 5000
Containerisation	Docker + Docker Compose	Swarm mode for multi-node
CI/CD	GitHub Actions	lint → test → build → push → deploy
Monitoring	Prometheus + Grafana	alert rules in `prometheus.yml`
Model Serving	FastAPI + Uvicorn	async endpoints, background workers
Distributed Training	torchrun (PyTorch DDP)	nproc_per_node = num GPUs

10. Repository & Project Structure

```
scene_reconstruction/
├── api/                        # FastAPI service
│   ├── main.py                 # /health, /ready, /reconstruct
endpoints
│   ├── models.py               # Pydantic request/response schemas
│   └── Dockerfile
├── model_server/               # Model loading & inference
│   ├── server.py               # HTTP wrapper around IMC2025Pipeline
│   └── Dockerfile
├── monitoring/                 # Prometheus + Grafana configs
│   ├── prometheus.yml
│   └── grafana/dashboards/
├── pipelines/                  # Core ML pipeline code
│   ├── imc2025_pipeline.py     # IMC2025Pipeline
│   ├── imc2025_mast3r_pipeline.py
│   ├── scene.py                # Scene dataclass
│   ├── matching.py             # run_point_tracking_matching, etc.
│   └── verification.py
├── matchers/                   # MASt3R, LightGlue, DUSt3R matchers
├── shortlists/                 # Retrieval-based shortlist generators
├── global_descriptors/         # DINOv2, ISC, SPoC, SigLIP2
├── colmap.py                   # COLMAP DB manipulation
├── data.py                     # DataSchema implementations
├── kernel.py                   # Entrypoint: run() and
run_and_save_submission()
├── evaluate_imc2025.py         # Evaluation + MLflow logging
├── conf/
│   ├── models.yaml             # Model path registry (local + kernel)
│   └── pipeline/imc2025/      # YAML configs per experiment
├── tests/
│   ├── unit/
│   ├── integration/
│   └── e2e/
├── docker-compose.yml
├── pyproject.toml
├── dvc.yaml                    # DVC pipeline definition
├── dvc.lock
└── .github/workflows/
    └── ci.yml                  # GitHub Actions CI/CD
```

---

11. CI/CD Pipeline

The GitHub Actions workflow (`ci.yml`) runs on every push to `main` and on pull requests:

Stage 1 — Lint & Type Check

```
- run: ruff check . && mypy .
```

Stage 2 — Unit Tests

```
- run: pytest tests/unit/ -v --cov=pipelines --cov-report=xml
```

Covers `Scene` dataclass, `Storage` classes, config validation, and RANSAC wrappers. No GPU required.

Stage 3 — Integration Tests

```
- run: pytest tests/integration/ -v
```

Tests end-to-end flow on a 5-image mini-scene using CPU-only mode.

Stage 4 — Docker Build & Smoke Test

```
- run: docker compose build
- run: docker compose up -d
- run: curl -f http://localhost:8000/health
- run: curl -f http://localhost:8001/ready
```

Stage 5 — DVC Repro (nightly only)

```
- run: dvc repro evaluate
```

Runs full evaluation on validation split and pushes metrics to DVC remote.

Stage 6 — Deploy Only triggers on tag push (`v*.*.*`):

```
- run: docker service update --image scene_api:$TAG scene_api
```

Includes automatic rollback if health check fails within 60 seconds.

---

12. Security & Compliance

Area	Measure
Data at rest	AES-256 encryption on model weight volumes
Data in transit	TLS 1.3 on all inter-container and external API communication
API authentication	JWT bearer tokens with short expiry (15 min)
Secrets management	Docker Secrets for API keys; never stored in environment variables or code
Image scanning	Trivy scans all Docker images in CI before push

Area	Measure
Dependency auditing	`pip-audit` runs in CI on every `pyproject.toml` change
Access logging	All API requests logged with request ID, timestamp, and anonymized client IP
Research-only models	MagicLeap SuperPoint is gated behind an internal license check; not exposed externally

Document version 1.0 — Aligned with IMC25 1st-place solution codebase and MLOps best practices guidelines.